

14

Come Play with Us! – Multiplayer Subtleties

Lots of great things in this world have incredibly humble beginnings. The contents of this book, from cover to cover, tell a story of a journey that began with nothing more than an interest and the will to create. Now that we're at the climax of our tale, why not end with a bang? Let's bring things back full circle and combine the framework we've developed with capabilities of networking to bring the third project of this book into new light! Let's connect our players not just through means of simple information exchange, but also through gameplay.

In this chapter, we're going to cover:

- Building a game server that supports previously implemented mechanics
- Exchanging entity data over a network
- Transforming existing game code into a client application
- Implementing player versus player combat
- Hiding network latency by smoothing out entity movement

There's a lot of code to cover, so let's get started!

Use of copyrighted resources

As always, it's fair to credit the artists who made the amazing graphics and sound effects that we're going to be using for our final project:

- *Simple small pixel hearts* by C.Nilsson under the CC-BY-SA 3.0 license: <http://opengameart.org/content/simple-small-pixel-hearts>
- *Grunt* by n3b under the CC-BY 3.0 license: <http://opengameart.org/content/grunt>

- *Swishes sound pack* by *artisticdude* under the CC0 license (public domain):
<http://opengameart.org/content/swishes-sound-pack>
- *3 Item sounds* by *Michel Baradari* under the CC-BY 3.0 license:
<http://opengameart.org/content/3-item-sounds>

Shared code

Since there are many instances where code we write is going to exist on both the client and the server side, let's discuss that first, starting with the way data exchange between both sides is made.

The most important part of our information exchange is updating entities on any and all connected clients. We do this by sending specialized structures back and forth, which contains relevant entity information. From now on, these structures are going to be referred to as snapshots. Let's see how they can be implemented, by taking a look at the `EntitySnapshot.h` file:

```
struct EntitySnapshot{
    std::string m_type;
    sf::Vector2f m_position;
    sf::Int32 m_elevation;
    sf::Vector2f m_velocity;
    sf::Vector2f m_acceleration;
    sf::Uint8 m_direction;
    sf::Uint8 m_state;
    sf::Uint8 m_health;
    std::string m_name;
};
```

The information we're going to be updating constantly for any given entity consists of its position and elevation, velocity, acceleration, the direction it's facing as well as the state it's in, and the entity's health and name. The type of an entity is also sent in a snapshot and used when creating the entity on the client side.



In this example, the order of the data members in the `EntitySnapshot` structure may not be the most efficient. Ordering data in your structures from biggest to smallest can help reduce their size, and therefore the bandwidth overhead. Structure alignment and packing are not going to be covered here, but it's a worthy subject to look into.

Something that helps a great deal in making our code more readable is overloading the *bitwise shift* operators of `sf::Packet` to support custom data types, such as `EntitySnapshot`:

```
sf::Packet& operator <<(sf::Packet& l_packet,
    const EntitySnapshot& l_snapshot);
sf::Packet& operator >>(sf::Packet& l_packet,
    EntitySnapshot& l_snapshot);
```

Actual implementation of these overloads exists inside the `EntitySnapshot.cpp` file:

```
sf::Packet& operator <<(sf::Packet& l_packet,
    const EntitySnapshot& l_snapshot)
{
    return l_packet << l_snapshot.m_type << l_snapshot.m_name
        << l_snapshot.m_position.x << l_snapshot.m_position.y
        << l_snapshot.m_elevation << l_snapshot.m_velocity.x
        << l_snapshot.m_velocity.y << l_snapshot.m_acceleration.x
        << l_snapshot.m_acceleration.y << l_snapshot.m_direction
        << l_snapshot.m_state << l_snapshot.m_health;
}

sf::Packet& operator >>(sf::Packet& l_packet,
    EntitySnapshot& l_snapshot)
{
    return l_packet >> l_snapshot.m_type >> l_snapshot.m_name
        >> l_snapshot.m_position.x >> l_snapshot.m_position.y
        >> l_snapshot.m_elevation >> l_snapshot.m_velocity.x
        >> l_snapshot.m_velocity.y >> l_snapshot.m_acceleration.x
        >> l_snapshot.m_acceleration.y >> l_snapshot.m_direction
        >> l_snapshot.m_state >> l_snapshot.m_health;
}
```

Other data exchanges are going to be more specific to the situation, so we're going to cover them later on. One thing we can do to prepare for that now, however, is updating the `Network` enumeration in the `NetworkDefinitions.h` file with a new value that is going to be used as a delimiter between different types of data in a specific packet:

```
enum class Network{
    HighestTimestamp = 2147483647, ClientTimeout = 10000,
    ServerPort = 5600, NullID = -1, PlayerUpdateDelim = -1
};
```

As we're going to work with the specific packet type that uses this delimiter on both the client and the server, its place is within the shared code space.

Additional components

First and foremost, entities that need to be synchronized between the server and client need to be marked and assigned a unique identifier. This is where the `C_Client` component comes in:

```
class C_Client : public C_Base{
public:
    C_Client(): C_Base(Component::Client),
        m_clientID((ClientID)Network::NullID){}
    void ReadIn(std::stringstream& l_stream){}

    ClientID GetClientID()const{ return m_clientID; }
    void SetClientID(const ClientID& l_id){ m_clientID = l_id; }
private:
    ClientID m_clientID;
};
```

It would also be nice to support entity names, in order to be able to store player nicknames. This can be accomplished by implementing a name component:

```
class C_Name : public C_Base{
public:
    C_Name() : C_Base(Component::Name){}
    void ReadIn(std::stringstream& l_stream){ l_stream >> m_name; }
    const std::string& GetName()const{ return m_name; }
    void SetName(const std::string& l_name){ m_name = l_name; }
private:
    std::string m_name;
};
```

It's a common thing in games to have a small cooldown period where an entity cannot be attacked, possibly also defining how long its hurting/death animation should last. Components that allow and define such functionality would ideally have to inherit from a base class, which simplifies the process of timing such events:

```
class C_TimedComponentBase : public C_Base{
public:
    C_TimedComponentBase(const Component& l_type)
        : C_Base(l_type), m_duration(sf::milliseconds(0)){}
    virtual ~C_TimedComponentBase(){}

    const sf::Time& GetTimer()const{ return m_duration; }
    void SetTimer(const sf::Time& l_time){ m_duration = l_time; }
    void AddToTimer(const sf::Time& l_time){ m_duration += l_time; }
```

```

    void Reset(){ m_duration = sf::milliseconds(0); }
protected:
    sf::Time m_duration;
};

```

One component that will make use of the base timed class will be C_Health:

```

using Health = unsigned int;

class C_Health : public C_TimedComponentBase{
public:
    C_Health(): C_TimedComponentBase(Component::Health),
        m_hurtDuration(0), m_deathDuration(0){}
    void ReadIn(std::stringstream& l_stream){
        l_stream >> m_maxHealth >> m_hurtDuration >> m_deathDuration;
        m_health = m_maxHealth;
    }
    Health GetHealth()const{ return m_health; }
    Health GetMaxHealth()const{ return m_maxHealth; }
    void SetHealth(const Health& l_health){ m_health = l_health; }
    void ResetHealth(){ m_health = m_maxHealth; }

    sf::Uint32 GetHurtDuration(){ return m_hurtDuration; }
    sf::Uint32 GetDeathDuration(){ return m_deathDuration; }
private:
    Health m_health;
    Health m_maxHealth;
    sf::Uint32 m_hurtDuration;
    sf::Uint32 m_deathDuration;
};

```

As you can see, it holds values for the current entity health, its maximum health value, and a few data members that hold the expected duration of being hurt and dying.

Naturally, we're going to need more entity message and event types in order to express the process of combat. The newly added types are highlighted in the following code snippet:

```

enum class EntityMessage{
    Move, Is_Moving, Frame_Change, State_Changed,
    Direction_Changed, Switch_State, Attack,
    Being_Attacked, Hurt, Die, Respawn, Removed_Entity
};
enum class EntityEvent{
    Spawned, Despawned, Colliding_X, Colliding_Y,

```

```
Moving_Left, Moving_Right, Moving_Up, Moving_Down,  
Elevation_Change, Became_Idle, Began_Moving, Began_Attacking  
};
```

The `EntityManager` class is also going to be shared between both sides. Some adjustments have to be made to its `AddEntity` and `RemoveEntity` methods in order to let the rest of the entity component system know when an entity has been added or removed:

```
int EntityManager::AddEntity(const Bitmask& l_mask, int l_id){  
    ...  
    m_systems->EntityModified(entity, l_mask);  
    m_systems->AddEvent(entity, (EventID)EntityEvent::Spawned);  
    return entity;  
}  
  
bool EntityManager::RemoveEntity(const EntityId& l_id){  
    ...  
    Message msg((MessageType)EntityMessage::Removed_Entity);  
    msg.m_receiver = l_id;  
    msg.m_int = l_id;  
    m_systems->GetMessageHandler()->Dispatch(msg);  
    ... // Removing all components.  
}
```

A lot more of the code we've written in previous chapters is actually going to need to be shared as well. Some of these classes, like the entity manager for example, have been slightly modified to serve as parents for derivatives of client and server implementations. We're not going to discuss that at length here, as the code files of this chapter should be more than helpful for familiarizing yourself with the code structure.

Building our game server

In *Chapter 13, We Have Contact! – Networking Basics*, we took a look at a very basic chat service that was supported by a server application and multiple clients connecting to it. Building a game server is quite similar to that. We have a piece of software that acts as a central point of interest to all its clients by doing all of the calculations and sending the results back to them in order to ensure proper and identical simulation is taking place across the board. Naturally, since we're not simply exchanging text messages, there is going to be a lot more data being sent back and forth, as well as more calculation on the server side.

First, we need to decide on a time interval value of sending entity snapshots. It has to be often enough to maintain smooth updates, but send as little information as possible to remain efficient. After some testing and tweaking, a sweet spot can be found pretty easily. For this particular project, let's say that an entity snapshot will be sent once every 100 milliseconds, which will be defined in `NetSettings.h`:

```
#define SNAPSHOT_INTERVAL 100
```

Sending 10 snapshots a second is enough to keep the clients happy and the server maintaining a relatively low bandwidth.

Additions to the entity component system

The majority of combat logic is going to take place on the server side. In order to support entities attacking one another, we need a new component to work with the `C_Attacker`:

```
class C_Attacker : public C_TimedComponentBase{
public:
    C_Attacker(): C_TimedComponentBase(Component::Attacker),
        m_attacked(false), m_knockback(0.f), m_attackDuration(0){}
    void ReadIn(std::stringstream& l_stream){
        l_stream >> m_offset.x >> m_offset.y
            >> m_attackArea.width >> m_attackArea.height
            >> m_knockback >> m_attackDuration;
    }

    void SetAreaPosition(const sf::Vector2f& l_pos){
        m_attackArea.left = l_pos.x;
        m_attackArea.top = l_pos.y;
    }

    const sf::FloatRect& GetAreaOfAttack(){ return m_attackArea; }
    const sf::Vector2f& GetOffset(){ return m_offset; }
    bool HasAttacked(){ return m_attacked; }
    void SetAttacked(bool l_attacked){ m_attacked = l_attacked; }
    float GetKnockback(){ return m_knockback; }
    sf::Uint32 GetAttackDuration(){ return m_attackDuration; }
private:
    sf::FloatRect m_attackArea;
    sf::Vector2f m_offset;
    bool m_attacked;
    float m_knockback;
    sf::Uint32 m_attackDuration;
};
```

The attacker component holds information about the size and location of the entity's attack area and possible offset, a flag to check if the entity has hit something while attacking, the force of knockback that is to be applied to another entity being attacked by this one, and the duration of the attack.

Implementing combat

Entity-on-entity combat is going to be a fairly simple addition, since we already have a nice collision system in place. It only requires a few additional lines of code inside the `EntityCollisions` method:

```
void S_Collision::EntityCollisions() {
    EntityManager* entities = m_systemManager->GetEntityManager();
    for (auto itr = m_entities.begin();
        itr != m_entities.end(); ++itr)
    {
        for (auto itr2 = std::next(itr);
            itr2 != m_entities.end(); ++itr2)
        {
            ...
            C_Attacker* attacker1 = entities->
                GetComponent<C_Attacker>(*itr, Component::Attacker);
            C_Attacker* attacker2 = entities->
                GetComponent<C_Attacker>(*itr2, Component::Attacker);
            if (!attacker1 && !attacker2) { continue; }
            Message msg((MessageType)EntityMessage::Being_Attacked);
            if (attacker1) {
                if (attacker1->GetAreaOfAttack().intersects(
                    collidable2->GetCollidable()))
                {
                    // Attacker-on-entity collision!
                    msg.m_receiver = *itr2;
                    msg.m_sender = *itr;
                    m_systemManager->GetMessageHandler()->Dispatch(msg);
                }
            }
            if (attacker2) {
                if (attacker2->GetAreaOfAttack().intersects(
                    collidable1->GetCollidable()))
                {
                    // Attacker-on-entity collision!
                    msg.m_receiver = *itr;
                    msg.m_sender = *itr2;
                    m_systemManager->GetMessageHandler()->Dispatch(msg);
                }
            }
        }
    }
}
```

```

    }
  }
}

```

First, both entities being checked get their attacker components fetched. If neither one of them has one, the iteration is skipped. Otherwise, a new message of type `Being_Attacked` is constructed. If the attacking entity's area of attach actually intersects the bounding box of another entity, this message is filled in with receiver and sender information and sent out.

In order to process and react properly to these collisions, as well as update all the entities that have potential to be in combat, we're going to need a new system: `S_Combat`! It doesn't have any additional methods other than the ones required to be implemented by the base system class, so there's really no need for us to examine its header. Let's take a look at its constructor and destructor instead:

```

S_Combat::S_Combat(SystemManager* l_systemMgr)
    : S_Base(System::Combat, l_systemMgr)
{
    Bitmask req;
    req.TurnOnBit((unsigned int)Component::Position);
    req.TurnOnBit((unsigned int)Component::Movable);
    req.TurnOnBit((unsigned int)Component::State);
    req.TurnOnBit((unsigned int)Component::Health);
    m_requiredComponents.push_back(req);
    req.ClearBit((unsigned int)Component::Health);
    req.TurnOnBit((unsigned int)Component::Attacker);
    m_requiredComponents.push_back(req);

    m_systemManager->GetMessageHandler()->
        Subscribe(EntityMessage::Being_Attacked, this);
}

```

This system will hold any entity that has position, is a movable entity with a state, and has a health component or attack component or both. It also subscribes to the `Being_Attacked` message in order to process attack area collisions.

Naturally, the same attack area cannot be positioned identically for all four directions that an entity is facing. Consider the following example:



Repositioning the attack area for each entity based on its current direction is done inside the Update method of this system:

```
void S_Combat::Update(float l_dT){
    EntityManager* entities = m_systemManager->GetEntityManager();
    for (auto &entity : m_entities){
        C_Attacker* attack = entities->
            GetComponent<C_Attacker>(entity, Component::Attacker);
        if (!attack){ continue; }
        sf::Vector2f offset = attack->GetOffset();
        sf::FloatRect AoA = attack->GetAreaOfAttack();
        Direction dir = entities->GetComponent<C_Movable>
            (entity, Component::Movable)->GetDirection();
        sf::Vector2f position = entities->GetComponent<C_Position>
            (entity, Component::Position)->GetPosition();
        if (dir == Direction::Left){ offset.x -= AoA.width / 2; }
        else if (dir == Direction::Right){ offset.x += AoA.width / 2; }
        else if (dir == Direction::Up){ offset.y -= AoA.height / 2; }
        else if (dir == Direction::Down){ offset.y += AoA.height / 2; }
        position -= sf::Vector2f(AoA.width / 2, AoA.height / 2);
        attack->SetAreaPosition(position + offset);
    }
}
```

If the current entity being checked has no `C_Attacker` component, the iteration is simply skipped. Otherwise, both the entity's area of attack and its offset are obtained, in addition to its current direction and position. In order to first center the attack area, half of its width and height is subtracted from the entity's position. The offset is then adjusted based on the direction the entity is facing, and the area of attack is moved to the latest position with it applied.

Let's take a look at a possible response to the message our collision system sends out:

```
void S_Combat::Notify(const Message& l_message){
    if (!HasEntity(l_message.m_receiver) ||
        !HasEntity(l_message.m_sender))
    {
        return;
    }
    EntityManager* entities = m_systemManager->GetEntityManager();
    EntityMessage m = (EntityMessage)l_message.m_type;
    switch (m){
    case EntityMessage::Being_Attacked:
        C_Health* victim = entities->GetComponent<C_Health>
            (l_message.m_receiver, Component::Health);
        C_Attacker* attacker = entities->GetComponent<C_Attacker>
            (l_message.m_sender, Component::Attacker);
        if (!victim || !attacker){ return; }
        S_State* StateSystem = m_systemManager->
            GetSystem<S_State>(System::State);
        if (StateSystem->GetState(l_message.m_sender) !=
            EntityState::Attacking)
        {
            return;
        }
        if (attacker->HasAttacked()){ return; }
        // Begin attacking.
        victim->SetHealth((victim->GetHealth() > 1 ?
            victim->GetHealth() - 1 : 0));
        attacker->SetAttacked(true);
        if (!victim->GetHealth()){
            StateSystem->ChangeState(l_message.m_receiver,
                EntityState::Dying, true);
        } else {
            Message msg((MessageType)EntityMessage::Hurt);
            msg.m_receiver = l_message.m_receiver;
            m_systemManager->GetMessageHandler()->Dispatch(msg);
        }

        // Knockback.
        Direction attackerDirection =entities->GetComponent<C_Movable>
            (l_message.m_sender, Component::Movable)->GetDirection();
        float Knockback = attacker->GetKnockback();
        sf::Vector2f KnockbackVelocity;
        if (attackerDirection == Direction::Left ||
```

```
        attackerDirection == Direction::Up)
    {
        Knockback = -Knockback;
    }
    if (attackerDirection == Direction::Left ||
        attackerDirection == Direction::Right)
    {
        KnockbackVelocity.x = Knockback;
    }
    else{ KnockbackVelocity.y = Knockback; }
    entities->GetComponent<C_Movable>
        (l_message.m_receiver, Component::Movable)->
        SetVelocity(KnockbackVelocity);
    break;
}
}
```

First, we check if the combat system has both the sender and receiver of this message. If that's the case, the `Being_Attacked` message is processed by first obtaining the health component of the entity being attacked, as well as the attacker component of the aggressor. The state of the attacking entity is then checked. If it is not currently attacking or if the entity has already attacked something else, the method is terminated by returning. Otherwise, the attack is initiated by first reducing the victim's health by 1. The attacker is then flagged for having already attacked an entity. If the victim's health is at value 0, its state is changed to `Dying`. Otherwise, a `Hurt` message is dispatched.

The remaining few lines of code deal with the victim entity being knocked back slightly, as it's being attacked. Both the attacker's direction and knockback force are obtained and a `sf::Vector2f` variable, signifying the applied force is created. If the attacker is facing either left or up, the knockback value is inverted. Also, if the attacking entity is facing either left or right, the knockback is applied on the *X* axis. Otherwise, the *Y* axis is used. Then, the force is simply applied as velocity through the victim's `C_Movable` component.

Server action timing

One major difference between running the same code on a server and a client is how certain actions and events are timed. Since we have no animations happening, there is no way to simply check when the last frame was reached and terminate an attack or death, for example. This is where manually setting certain time values comes in. For this, we're going to need the `S_Timers` system. Since it also doesn't have any additional methods other than the required ones, the class definition is not necessary.

Let's start by taking a look at the constructor and destructor of this system:

```
S_Timers::S_Timers(SystemManager* l_systemMgr)
: S_Base(System::Timers, l_systemMgr)
{
    Bitmask req;
    req.TurnOnBit((unsigned int)Component::State);
    req.TurnOnBit((unsigned int)Component::Attacker);
    m_requiredComponents.push_back(req);
    req.ClearBit((unsigned int)Component::Attacker);
    req.TurnOnBit((unsigned int)Component::Health);
    m_requiredComponents.push_back(req);
}
```

Once again, we simply subscribe to the state component as well as the attacker component, health component, or both. Nothing interesting happens here, so let's move on to the Update method that makes timing on the server side possible:

```
void S_Timers::Update(float l_dT){
    EntityManager* entities = m_systemManager->GetEntityManager();
    for (auto &entity : m_entities){
        EntityState state = entities->GetComponent<C_State>
            (entity, Component::State)->GetState();
        if (state == EntityState::Attacking){
            C_Attacker* attack = entities->GetComponent<C_Attacker>
                (entity, Component::Attacker);
            attack->AddToTimer(sf::seconds(l_dT));
            if (attack->GetTimer().asMilliseconds() <
                attack->GetAttackDuration())
            {
                continue;
            }
            attack->Reset();
            attack->SetAttacked(false);
        } else if (state == EntityState::Hurt ||
            state == EntityState::Dying)
        {
            C_Health* health = entities->
                GetComponent<C_Health>(entity, Component::Health);
            health->AddToTimer(sf::seconds(l_dT));
            if ((state == EntityState::Hurt &&
                health->GetTimer().asMilliseconds() <
                health->GetHurtDuration()) ||
                (state == EntityState::Dying &&
```

```
        health->GetTimer().asMilliseconds() <
        health->GetDeathDuration())
    {
        continue;
    }

    health->Reset();
    if (state == EntityState::Dying){
        Message msg((MessageType)EntityMessage::Respawn);
        msg.m_receiver = entity;
        m_systemManager->GetMessageHandler()->Dispatch(msg);
        health->ResetHealth();
    }
    else { continue; }
    m_systemManager->GetSystem<S_State>(System::State)->
        ChangeState(entity, EntityState::Idle, true);
}
}
```

In this system, both the attacker and health components are checked to see whether they have reached specific time values that are provided in the entity file. If the entity is in an attacking state, the attacker component is obtained and the elapsed time is added to it. If the attack duration is passed, the timer is reset and the "attacked" flag is set back to false, making another attack possible.

If the entity is in either the hurting or dying state, the respectful time values are checked against predetermined durations and the timer is reset once again. If the entity is actually in a dying state, a Respawn message is sent out as well, in order to reset its animation, health and move the entity to a specific location where it "respawns".

Server network system

Handling entity networking on the server side can be made a lot easier by simply building a dedicated system that would already have access to entity data by design. This is where the server network system comes in.

Let's start with how entities are going to be controlled by players. In previous chapters, we simply used messaging to move entities around on the client side. Obviously, due to network delays and bandwidth restrictions, it would be problematic to simply send out a bunch of messages whenever a client moves. It's much more efficient to simply keep track of a player's input state, as this simple structure demonstrates:

```
struct PlayerInput{
    int m_movedX;
    int m_movedY;
```

```

    bool m_attacking;
    PlayerInput() : m_movedX(0), m_movedY(0), m_attacking(false) {}
};

using PlayerInputContainer = std::unordered_map<EntityId,
    PlayerInput>;

```

The first two data members will contain the amount of times a player has been moved along either axis. On the client side, we're going to be sending input states to the server at a specific interval, which means we have the benefit of combining messages into neat packets, as well as process out redundant movement, such as moving left and right by the same amount. Additionally, clients are also going to be sending their attacking states. All of this information will be held in a container, which tethers it to a specific entity ID.

Now, let's take a look at the header file of the network system we're going to implement:

```

class S_Network : public S_Base{
public:
    S_Network(SystemManager* l_systemMgr);
    ~S_Network();

    void Update(float l_dT);
    void HandleEvent(const EntityId& l_entity,
        const EntityEvent& l_event);
    void Notify(const Message& l_message);

    bool RegisterClientID(const EntityId& l_entity,
        const ClientID& l_client);
    void RegisterServer(Server* l_server);
    ClientID GetClientID(const EntityId& l_entity);
    EntityId GetEntityID(const ClientID& l_client);

    void CreateSnapshot(sf::Packet& l_packet);
    void UpdatePlayer(sf::Packet& l_packet, const ClientID& l_cid);
private:
    PlayerInputContainer m_playerInput;
    Server* m_server;
};

```

As usual, we have the required methods implemented, as well as a few additional ones. Since we're going to link behavior between clients and entities, we have a few methods that help us register and obtain this relationship information. On top of that, a few helper methods exist for creating snapshots of entity states and updating a specific client's information from an incoming packet.

Implementing the network system

Let's start with the constructor and destructor of the network system:

```
S_Network::S_Network(SystemManager* l_systemMgr)
: S_Base(System::Network, l_systemMgr)
{
    Bitmask req;
    req.TurnOnBit((unsigned int)Component::Client);
    m_requiredComponents.push_back(req);

    MessageHandler* messageHandler =
        m_systemManager->GetMessageHandler();
    messageHandler->Subscribe(EntityMessage::Removed_Entity, this);
    messageHandler->Subscribe(EntityMessage::Hurt, this);
    messageHandler->Subscribe(EntityMessage::Respawn, this);
}
```

This particular system is only going to require a single component: `C_Client`. It also subscribes to entity removal, hurt, and respawn messages.

Next, the Update method:

```
void S_Network::Update(float l_dT){
    EntityManager* entities = m_systemManager->GetEntityManager();
    for (auto &entity : m_entities){
        auto& player = m_playerInput[entity];
        if (player.m_movedX || player.m_movedY){
            if (player.m_movedX){
                Message msg((MessageType)EntityMessage::Move);
                msg.m_receiver = entity;
                if (player.m_movedX > 0){msg.m_int=(int)Direction::Right;}
                else { msg.m_int = (int)Direction::Left; }
                m_systemManager->GetMessageHandler()->Dispatch(msg);
            }

            if (player.m_movedY){
                Message msg((MessageType)EntityMessage::Move);
                msg.m_receiver = entity;
            }
        }
    }
}
```

```

        if (player.m_movedY > 0){msg.m_int=(int)Direction::Down;}
        else { msg.m_int = (int)Direction::Up; }
        m_systemManager->GetMessageHandler()->Dispatch(msg);
    }
}
if (player.m_attacking){
    Message msg((MessageType)EntityMessage::Attack);
    msg.m_receiver = entity;
    m_systemManager->GetMessageHandler()->Dispatch(msg);
}
}
}

```

This is where we process the current control state of a client and apply it to the entity. Relevant messages are constructed and sent out, based on the state of a client's input.

Next, let's deal with those three message types that this system is subscribed to:

```

void S_Network::Notify(const Message& l_message){
    if (!HasEntity(l_message.m_receiver)){ return; }
    EntityMessage m = EntityMessage(l_message.m_type);
    if (m == EntityMessage::Removed_Entity){
        m_playerInput.erase(l_message.m_receiver);
        return;
    }
    if (m == EntityMessage::Hurt){
        sf::Packet packet;
        StampPacket(PacketType::Hurt, packet);
        packet << l_message.m_receiver;
        m_server->Broadcast(packet);
        return;
    }
    if (m == EntityMessage::Respawn){
        C_Position* position = m_systemManager->GetEntityManager()->
            GetComponent<C_Position>(l_message.m_receiver,
                Component::Position);
        if (!position){ return; }
        position->SetPosition(64.f, 64.f);
        position->SetElevation(1);
    }
}

```

First, if the entity is being removed, the player input information of the corresponding client in control of that entity gets erased. If a message about an entity getting hurt is received, a hurt packet is constructed and sent to all clients to notify them of an entity taking damage. Lastly, an entity respawn message is handled by resetting its position and elevation to some pre-defined values. These coordinates can easily be randomized or read in from the map file, but for demonstration purposes this works just fine.

When a client connects to our server and an entity for it is created, we need to have a method that allows us to express that relationship by binding the two values together, as shown here:

```
bool S_Network::RegisterClientID(const EntityId& l_entity,
    const ClientID& l_client)
{
    if (!HasEntity(l_entity)){ return false; }
    m_systemManager->GetEntityManager()->GetComponent<C_Client>
        (l_entity, Component::Client)->SetClientID(l_client);
    return true;
}
```

Since we're going to be storing the client ID inside the client component, it's obtained through the entity manager and used in exactly that manner.

The network class is also going to need access to an instance of the Server class, hence the following method:

```
void S_Network::RegisterServer(Server* l_server){
    m_server = l_server;
}
```

Next, a few methods of convenience for obtaining client and entity IDs:

```
ClientID S_Network::GetClientID(const EntityId& l_entity){
    if (!HasEntity(l_entity)){ return (ClientID)Network::NullID; }
    return m_systemManager->GetEntityManager()->
        GetComponent<C_Client>(l_entity, Component::Client)->
        GetClientID();
}
EntityId S_Network::GetEntityID(const ClientID& l_client){
    EntityManager* e = m_systemManager->GetEntityManager();
    auto entity = std::find_if(m_entities.begin(), m_entities.end(),
        [&e, &l_client](EntityId& id){
            return e->GetComponent<C_Client>
```

```

        (id, Component::Client)->GetClientID() == l_client;
    });
    return(entity != m_entities.end() ?
        *entity : (EntityId)Network::NullID);
}

```

Snapshot creation itself also deserves its own method:

```

void S_Network::CreateSnapshot(sf::Packet& l_packet){
    sf::Lock lock(m_server->GetMutex());
    ServerEntityManager* e =
        (ServerEntityManager*)m_systemManager->GetEntityManager();
    StampPacket(PacketType::Snapshot, l_packet);
    l_packet << sf::Int32(e->GetEntityCount());
    if (e->GetEntityCount()){
        e->DumpEntityInfo(l_packet);
    }
}

```

Because we're accessing entity information that could be changed, the server mutex has to be locked before we access it. After assigning a snapshot type to the packet provided as an argument, we write the number of entities into it as well. If the number is above zero, a `DumpEntityInfo` method is invoked. This method is defined within our `ServerEntityManager` class and will be covered shortly.

Lastly, let's handle the incoming player update packets:

```

void S_Network::UpdatePlayer(sf::Packet& l_packet,
    const ClientID& l_cid)
{
    sf::Lock lock(m_server->GetMutex());
    EntityId eid = GetEntityID(l_cid);
    if (eid == -1){ return; }
    if (!HasEntity(eid)){ return; }
    sf::Int8 entity_message;
    m_playerInput[eid].m_attacking = false;
    while (l_packet >> entity_message){
        switch (entity_message){
            case sf::Int8(EntityMessage::Move):
            {
                sf::Int32 x = 0, y = 0;
                l_packet >> x >> y;
                m_playerInput[eid].m_movedX = x;
                m_playerInput[eid].m_movedY = y;
                break;
            }
        }
    }
}

```

```
    }
    case sf::Int8(EntityMessage::Attack) :
    {
        sf::Int8 attackState;
        l_packet >> attackState;
        if (attackState) { m_playerInput[eid].m_attacking = true; }
        break;
    }
}
sf::Int8 delim = 0;
if (!(l_packet >> delim) || delim !=
    (sf::Int8)Network::PlayerUpdateDelim)
{
    std::cout << "Faulty update!" << std::endl;
    break;
}
}
```

Before anything can be done, we must make sure that the server mutex is locked and the client sending this packet has a valid entity attached to it. This is done by obtaining the entity ID and checking its validity in the next two lines. A local variable named `entity_message` is then created in order to hold the message type that the client is going to be sending to us. The attack state of the entity is then set to `false` by default and iterating over the packet's information begins.

Encountering a `Move` message is dealt with by extracting the X and Y values from the packet and overwriting our player movement information for the given entity with them. The `Attack` message has one less value to worry about. The player's `m_attacking` flag is set to `true` if the incoming player state contains anything else but zero.

Server entity and system management

The components and systems supported on the server side are obviously going to differ from those on the client side. On top of that, custom methods for both ends help out a great deal by allowing the base class to remain unmodified, while the derivatives can deal with side-specific logic. Let's take a look at our simple extension to the `EntityManager` class that runs on the server side:

```
ServerEntityManager::ServerEntityManager(SystemManager* l_sysMgr)
: EntityManager(l_sysMgr)
{
    AddComponentType<C_Position>(Component::Position);
}
```

```

AddComponentType<C_State>(Component::State);
AddComponentType<C_Movable>(Component::Movable);
AddComponentType<C_Controller>(Component::Controller);
AddComponentType<C_Collidable>(Component::Collidable);
AddComponentType<C_Client>(Component::Client);
AddComponentType<C_Health>(Component::Health);
AddComponentType<C_Name>(Component::Name);
AddComponentType<C_Attacker>(Component::Attacker);
}

```

We're obviously not going to need any graphics or sound related component types here. It's the client's job to deal with those.

This class will also be useful when creating entity snapshots. All of the entity information is dumped into a provided instance of `sf::Packet` by using this method:

```

void ServerEntityManager::DumpEntityInfo(sf::Packet& l_packet){
    for (auto &entity : m_entities){
        l_packet << sf::Int32(entity.first);
        EntitySnapshot snapshot;
        snapshot.m_type = entity.second.m_type;
        const auto& mask = entity.second.m_bitmask;
        if (mask.GetBit((unsigned int)Component::Position)){
            C_Position* p = GetComponent<C_Position>(entity.first,
                Component::Position);
            snapshot.m_position = p->GetPosition();
            snapshot.m_elevation = p->GetElevation();
        }
        if (mask.GetBit((unsigned int)Component::Movable)){
            C_Movable* m = GetComponent<C_Movable>(entity.first,
                Component::Movable);
            snapshot.m_velocity = m->GetVelocity();
            snapshot.m_acceleration = m->GetAcceleration();
            snapshot.m_direction = sf::Uint8(m->GetDirection());
        }
        if (mask.GetBit((unsigned int)Component::State)){
            C_State* s = GetComponent<C_State>(entity.first,
                Component::State);
            snapshot.m_state = sf::Uint8(s->GetState());
        }
        if (mask.GetBit((unsigned int)Component::Health)){
            C_Health* h = GetComponent<C_Health>(entity.first,
                Component::Health);
            snapshot.m_health = h->GetHealth();
        }
    }
}

```

```
    }
    if (mask.GetBit((unsigned int)Component::Name)) {
        C_Name* n = GetComponent<C_Name>(entity.first,
            Component::Name);
        snapshot.m_name = n->GetName();
    }
    l_packet << snapshot;
}
}
```

The entity ID is written into the packet instance first. An `EntitySnapshot` variable is created afterwards, and it is filled with relevant component information, provided these components exist at all. Once that's done, the snapshot instance is written to the packet, which is made incredibly easy thanks to its overloaded `<<` and `>>` operators.

For system management on the server side, we only need to handle the systems that are added:

```
ServerSystemManager::ServerSystemManager() {
    AddSystem<S_Network>(System::Network);
    AddSystem<S_State>(System::State);
    AddSystem<S_Control>(System::Control);
    AddSystem<S_Movement>(System::Movement);
    AddSystem<S_Timers>(System::Timers);
    AddSystem<S_Collision>(System::Collision);
    AddSystem<S_Combat>(System::Combat);
}
```

Similar to what we did for components, we simply exclude anything graphical or sound related.

Main server class

Similar to the client side's `Game` class, a supervisor object is going to be needed on the server side as well. We're going to be keeping instances of the game map, entity, and server managers, and the `Server` class itself in a new class, simply called `World`. Let's start by taking a look at the header file:

```
class World{
public:
    World();
    ~World();

    void Update(const sf::Time& l_time);
    void HandlePacket(sf::IpAddress& l_ip,
```

```

        const PortNumber& l_port, const PacketID& l_id,
        sf::Packet& l_packet, Server* l_server);
void ClientLeave(const ClientID& l_client);
void CommandLine();

bool IsRunning();
private:
    sf::Time m_tpsTime;
    sf::Time m_serverTime;
    sf::Time m_snapshotTimer;
    sf::Thread m_commandThread;
    Server m_server;
    ServerSystemManager m_systems;
    ServerEntityManager m_entities;
    bool m_running;

    Map m_map;
    unsigned int m_tick;
    unsigned int m_tps;
};

```

Similar to Game, it has an `Update` method where all of the time-related magic is going to happen. It also has methods for handling custom packet types, handling a client leaving, and processing command-line input.

Data member wise, we're looking at a few `sf::Time` instances for keeping track of the current server time, as well as delivery time for snapshots. A `sf::Thread` instance for the command line is also quite handy to have around.

Last but not least, the `m_tpsTime`, `m_tick` and `m_tps` data members exist for the simple convenience of measuring the update rate on the server. The number of updates per second, also known as ticks, is quite useful for tracking down and resolving performance issues.

Implementing the world class

Let's kick this class into gear, starting with the constructor and destructor:

```

World::World(): m_server(&World::HandlePacket, this),
    m_commandThread(&World::CommandLine, this), m_entities(nullptr),
    m_map(&m_entities), m_tick(0), m_tps(0), m_running(false)
{
    if (!m_server.Start()){ return; }
    m_running = true;
    m_systems.SetEntityManager(&m_entities);
}

```

```
m_entities.SetSystemManager(&m_systems);
m_map.LoadMap("media/Maps/map1.map");
m_systems.GetSystem<S_Collision>(System::Collision)->
    SetMap(&m_map);
m_systems.GetSystem<S_Movement>(System::Movement)->
    SetMap(&m_map);
m_systems.GetSystem<S_Network>(System::Network)->
    RegisterServer(&m_server);
m_server.BindTimeoutHandler(&World::ClientLeave, this);
m_commandThread.launch();
}

World::~~World(){ m_entities.SetSystemManager(nullptr); }
```

Our `Server` instance is set up by providing a valid packet handler in the initializer list, where the command thread is also set up. In the actual body of the constructor, we first attempt to start the server and catch a possible failure in an `if` statement. Upon a successful start, the `m_running` flag is set to `true` and both the entity manager and system manager are provided with pointers to each other. The game map is then loaded and relevant systems are provided with its instance. After our network system is made available with an instance of `Server`, the `ClientLeave` method is fed in as the timeout handler and the command line thread is launched.

Upon destruction of the `World` class, all we really have to worry about is taking away the entity manager's access to the system manager.

Next, let's keep the action rolling by updating everything:

```
void World::Update(const sf::Time& l_time){
    if (!m_server.IsRunning()){ m_running = false; return; }
    m_serverTime += l_time;
    m_snapshotTimer += l_time;
    m_tpsTime += l_time;
    m_server.Update(l_time);
    m_server.GetMutex().lock();
    m_systems.Update(l_time.asSeconds());
    m_server.GetMutex().unlock();
    if (m_snapshotTimer.asMilliseconds() >= SNAPSHOT_INTERVAL){
        sf::Packet snapshot;
        m_systems.GetSystem<S_Network>(System::Network)->
            CreateSnapshot(snapshot);
        m_server.Broadcast(snapshot);
        m_snapshotTimer = sf::milliseconds(0);
    }
}
```

```

    if (m_tpsTime >= sf::milliseconds(1000)){
        m_tps = m_tick;
        m_tick = 0;
        m_tpsTime = sf::milliseconds(0);
    } else {
        ++m_tick;
    }
}

```

The server instance is first checked for having stopped. If that's the case, the world class itself is stopped and the `Update` method is returned from. Otherwise, all of our time values are updated alongside the server class. The system manager's `Update` method is then invoked in order to update all of the entity information. The server mutex has to be locked while that happens, as entity information could potentially be changed.

Once everything is up to date, the snapshot timer is checked to see if it has exceeded the snapshot interval. With that being the case, a snapshot packet is created and filled in by using the `CreateSnapshot` method of `S_Network`. The packet is then broadcasted to every single client and the snapshot timer is reset to zero.



Ticks Per Second (TPS) are measured by increasing the `m_tick` data member every update, provided the TPS timer hasn't exceeded one second. If that's the case, `m_tps` is assigned the value of `m_tick`, which in turn gets set back to zero, alongside the TPS timer.

Handling incoming packets is the next piece of the puzzle:

```

void World::HandlePacket(sf::IpAddress& l_ip,
    const PortNumber& l_port, const PacketID& l_id,
    sf::Packet& l_packet, Server* l_server)
{
    ClientID id = l_server->GetClientID(l_ip, l_port);
    PacketType type = (PacketType)l_id;
    if (id >= 0){
        if (type == PacketType::Disconnect){
            ClientLeave(id);
            l_server->RemoveClient(l_ip, l_port);
        } else if (type == PacketType::Message){
            // ...
        } else if (type == PacketType::PlayerUpdate){
            m_systems.GetSystem<S_Network>(System::Network)->
                UpdatePlayer(l_packet, id);
        }
    }
}

```

```
    } else {
        if (type != PacketType::Connect){ return; }
        std::string nickname;
        if (!(l_packet >> nickname)){ return; }
        ClientID cid = l_server->AddClient(l_ip, l_port);
        if (cid == -1){
            sf::Packet packet;
            StampPacket(PacketType::Disconnect, packet);
            l_server->Send(l_ip, l_port, packet);
            return;
        }
        sf::Lock lock(m_server.GetMutex());
        sf::Int32 eid = m_entities.AddEntity("Player");
        if (eid == -1){ return; }
        m_systems.GetSystem<S_Network>(System::Network)->
            RegisterClientID(eid, cid);
        C_Position* pos = m_entities.GetComponent<C_Position>
            (eid, Component::Position);
        pos->SetPosition(64.f, 64.f);
        m_entities.GetComponent<C_Name>(eid, Component::Name)->
            SetName(nickname);
        sf::Packet packet;
        StampPacket(PacketType::Connect, packet);
        packet << eid;
        packet << pos->GetPosition().x << pos->GetPosition().y;
        if (!l_server->Send(cid, packet)){
            std::cout << "Unable to respond to connect packet!"
                << std::endl;
            return;
        }
    }
}
```

The client ID is first obtained from the originating IP address and port number. If a client with that information exists, we're interested in three packet types that can be received from it. First, the client disconnect packet is handled by invoking the `ClientLeave` method with the client ID passed in as the only argument. Next, the actual client is removed from the server class.

The next packet type, `Message`, is left unimplemented for now. We're not going to send chat messages between clients just yet, but this is where it would be implemented in the future. Following that is the player update packet type, in which case the packet is simply passed into the network system to be processed. We have already covered this.

If the origin information of the incoming data does not yield us a valid client ID, we're only interested in communications that attempt to connect. First, we attempt to extract a string from the packet, which would be the player nickname. If that fails, this method is returned from. Next, the client information is added and its success is checked by analyzing the returned client ID. In case of a failure, a `Disconnect` packet is sent back to the originating source and the method is returned from. Otherwise, the server mutex is locked and we attempt to add a new player entity. Failure to do that, once again, would result in returning from this method. The client ID is then registered in the network system and the position of our newly added player entity is set to some pre-defined values. The name component of the player entity is also adjusted to reflect the entered nickname. At this point, a connect packet is constructed as a response. It contains the entity ID of the player, as well as its spawn position. The packet is then sent out to our new client.

Leaving the server is a much simpler procedure in comparison to this. Let's take a look:

```
void World::ClientLeave(const ClientID& l_client){
    sf::Lock lock(m_server.GetMutex());
    S_Network* network = m_systems.
        GetSystem<S_Network>(System::Network);
    m_entities.RemoveEntity(network->GetEntityID(l_client));
}
```

The server mutex gets locked before this operation is performed. The network system is then obtained and the `RemoveEntity` method of our entity manager is invoked with the return value of network system's `GetEntityID` method. This effectively removes the entity.

Implementing some basic commands on the server side proves to be more than useful. Let's take a look at a very basic setup of a command-line thread:

```
void World::CommandLine(){
    while (m_server.IsRunning()){
        std::string str;
        std::getline(std::cin, str);
        if (str == "terminate"){
            m_server.Stop();
            m_running = false;
            break;
        } else if (str == "disconnectall"){
            std::cout << "Disconnecting all clients..." << std::endl;
            m_server.DisconnectAll();
            sf::Lock lock(m_server.GetMutex());
            m_entities.Purge();
        }
    }
}
```

```
    } else if (str.find("tps") != std::string::npos){
        std::cout << "TPS: " << m_tps << std::endl;
    } else if (str == "clients"){
        std::cout << m_server.GetClientCount()
            << " clients online:" << std::endl;
        std::cout << m_server.GetClientList()
            << std::endl;
    } else if (str == "entities"){
        std::cout << "Current entity count: "
            << m_entities.GetEntityCount() << std::endl;
    }
}
```

First, a loop is entered and kept alive as long as the server is running. Next, the command line is prompted to obtain a line of input. The first command we process is "terminate". This stops the server and breaks out of the command line loop, which is helpful. The following command disconnects every single client and purges all entities that currently exist. Notice that the server mutex gets locked before the purge. The next command simply displays the current tick per second rate. Typing in "clients" would result in a list of clients currently connected that contains their IP addresses, port numbers, and latency values. Lastly, the "entities" command simply prints out the number of entities that are currently in the world.

The last and definitely the least interesting method is useful for obtaining the current status of the world:

```
bool World::IsRunning(){ return m_running; }
```

Server entry point

Now let's put all of this effort to work. The following are the contents of our `Server_Main.cpp` file:

```
#include "World.h"

int main(){
    World world;
    sf::Clock clock;
    clock.restart();

    while (world.IsRunning()){
        world.Update(clock.restart());
    }
    return 0;
}
```

It couldn't get simpler than this. A new instance of the `World` class is created, alongside a clock that is promptly restarted. Our main `while` loop is entered with the condition that the world instance has to keep running. It gets updated every iteration with the return value of `clock.restart()`. After the loop is terminated, zero is returned to end the program successfully.

All of this brings us a very nice looking and capable console window that's ready to handle some incoming connections:

```
Incoming port: 5600. Outgoing port: 50979
Beginning to listen...
--- Loading a map: media/Maps/map1.map
--- Map Loaded! ---
clients
2 clients online:
-----
ID      Client IP:PORT      Ping
-----
0       127.0.0.1:50980      0ms.
1       127.0.0.1:50981      0ms.
-----
Total data sent: 9kB. Total data received: 3kB
=
```

This by itself is, of course, completely useless without the client that draws all of the pretty images as a result of the server communicating with it. That is the next major task on our list.

Developing the game client

With proper backend support from the server, we can now focus entirely on client-side details and spoil ourselves a little with pretty visuals that always yield that sense of accomplishment a lot quicker than anything that runs in the background. Let's start by creating the client's own version of `NetSettings.h`:

```
#define NET_RENDER_DELAY 100 // ms.
#define PLAYER_UPDATE_INTERVAL 50 // ms
```

We have a couple of macros to work with here. First is the expected delay between what's being rendered on screen and real time. This means that technically we're going to be rendering all action about 100 milliseconds in the past. The second macro is the interval at which we're going to be sending updates to the server. 50 milliseconds gives us plenty of time to gather a few input states and let the server know what's going on.

Entity component system expansions

As in the case of the server, additional components and systems are necessary if we want to realize any of our goals. Unlike the server, however, these additions to the client entity component system are going to serve an entirely different purpose. It's going to be important for us to see the names and health values of all players in the game. We're going to shoot for something like this:



In order to easily maintain these glyphs floating above an entity, we're going to need a new type of component that describes exactly where they're supposed to be rendered:

```
class C_UI_Element : public C_Base{
public:
    C_UI_Element() : C_Base(Component::UI_Element),
        m_showHealth(false), m_showName(false){}
    void ReadIn(std::stringstream& l_stream){
        l_stream >> m_offset.x >> m_offset.y;
    }

    const sf::Vector2f& GetOffset(){ return m_offset; }
    void SetOffset(const sf::Vector2f& l_offset){ m_offset = l_offset; }

    void SetShowHealth(bool l_show){ m_showHealth = l_show; }
    void SetShowName(bool l_show){ m_showName = l_show; }
    bool ShowHealth(){ return m_showHealth; }
    bool ShowName(){ return m_showName; }
private:
    sf::Vector2f m_offset;
    bool m_showHealth;
    bool m_showName;
};
```

The `C_UI_Element` component will read in two offset values, one for X and one for Y, from the entity file. This way, characters of different sizes can define their own rules of where this information will appear. We also included a couple of Boolean flags in case the health or name information ever needs to be disabled for some reason.

The component alone isn't going to do anything fancy, so let's create a new system that actually makes something happen:

```
class S_CharacterUI : public S_Base{
public:
    S_CharacterUI(SystemManager* l_systemMgr);
    ~S_CharacterUI();

    void Update(float l_dT);
    void HandleEvent(const EntityId& l_entity,
        const EntityEvent& l_event);
    void Notify(const Message& l_message);

    void Render(Window* l_wind);
private:
    sf::Sprite m_heartBar;
    sf::Text m_nickname;
    sf::RectangleShape m_nickbg;
    sf::Vector2u m_heartBarSize;
};
```

Note that this system has a `Render` method. We're not only going to update the position of the graphical elements, but also draw them on the screen. This includes a sprite that will be bound to whatever texture is chosen to represent health, an instance of `sf::Text` that will hold the entity's name, a rectangle background that will be rendered behind the name, and a data member that holds the size of the health bar texture.

With that out of the way, let's start implementing this system!

```
S_CharacterUI::S_CharacterUI(SystemManager* l_systemMgr)
    : S_Base(System::Character_UI, l_systemMgr)
{
    Bitmask req;
    req.TurnOnBit((unsigned int)Component::Position);
    req.TurnOnBit((unsigned int)Component::UI_Element);
    req.TurnOnBit((unsigned int)Component::Health);
    m_requiredComponents.push_back(req);
    req.ClearBit((unsigned int)Component::Health);
```

```
req.TurnOnBit((unsigned int)Component::Name);
m_requiredComponents.push_back(req);

ClientSystemManager* mgr = (ClientSystemManager*)m_systemManager;
mgr->GetTextureManager()->RequireResource("HeartBar");
mgr->GetFontManager()->RequireResource("Main");
sf::Texture* txtr = mgr->GetTextureManager()->
    GetResource("HeartBar");
txtr->setRepeated(true);
m_heartBarSize = txtr->getSize();
m_heartBar.setTexture(*txtr);
m_heartBar.setScale(0.5f, 0.5f);
m_heartBar.setOrigin(m_heartBarSize.x / 2, m_heartBarSize.y);
m_nickname.setFont(*mgr->GetFontManager()->GetResource("Main"));
m_nickname.setCharacterSize(9);
m_nickname.setColor(sf::Color::White);
m_nickbg.setFillColor(sf::Color(100, 100, 100, 100));
}
```

The first order of business here is, of course, setting up the component requirements. An entity has to have a position component and a UI element component, in addition to some combination of the health and name components. The rest of the constructor is used to set up the texture and font resources for our graphics. Our health bar texture is set to be repeated so we can represent any health value. The actual texture is only the size of a single heart.

The resources for these elements obviously have to be released when they're no longer needed. That's where the destructor comes in:

```
S_CharacterUI::~S_CharacterUI() {
    ClientSystemManager* mgr =
        (ClientSystemManager*)m_systemManager;
    mgr->GetTextureManager()->ReleaseResource("HeartBar");
    mgr->GetFontManager()->ReleaseResource("Main");
}
```

Lastly, the most important part of this system is contained within the Render method:

```
void S_CharacterUI::Render(Window* l_wind) {
    EntityManager* entities = m_systemManager->GetEntityManager();
    for (auto &entity : m_entities) {
        C_Health* health = entities->
            GetComponent<C_Health>(entity, Component::Health);
        C_Name* name = entities->
            GetComponent<C_Name>(entity, Component::Name);
        C_Position* pos = entities->
```

```

        GetComponent<C_Position>(entity, Component::Position);
    C_UI_Element* ui = entities->
        GetComponent<C_UI_Element>(entity, Component::UI_Element);
    if (health){
        m_heartBar.setTextureRect(sf::IntRect(0, 0,
            m_heartBarSize.x * health->GetHealth(),
            m_heartBarSize.y));
        m_heartBar.setOrigin((
            m_heartBarSize.x * health->GetHealth())/2,
            m_heartBarSize.y);
        m_heartBar.setPosition(pos->GetPosition() +ui->GetOffset());
        l_wind->GetRenderWindow()->draw(m_heartBar);
    }
    if (name){
        m_nickname.setString(name->GetName());
        m_nickname.setOrigin(m_nickname.getLocalBounds().width / 2,
            m_nickname.getLocalBounds().height / 2);
        if (health){
            m_nickname.setPosition(m_heartBar.getPosition().x,
                m_heartBar.getPosition().y - (m_heartBarSize.y));
        } else {
            m_nickname.setPosition(pos->GetPosition() +
                ui->GetOffset());
        }
        m_nickbg.setSize(sf::Vector2f(
            m_nickname.getGlobalBounds().width + 2,
            m_nickname.getCharacterSize() + 1));
        m_nickbg.setOrigin(m_nickbg.getSize().x / 2,
            m_nickbg.getSize().y / 2);
        m_nickbg.setPosition(m_nickname.getPosition().x + 1,
            m_nickname.getPosition().y + 1);
        l_wind->GetRenderWindow()->draw(m_nickbg);
        l_wind->GetRenderWindow()->draw(m_nickname);
    }
}
}

```

For each entity, we obtain all four components that we're going to be working with. Since there can be instances where either the name or health components exist without the other one present, both of them must be checked before we commit to rendering them.

The health bar portion is drawn by first resetting the texture rectangle of the sprite. Its width is changed to the result of multiplying the width of a single heart in the texture by the health value an entity has. The Y value is left unchanged. The origin of the sprite is then changed to be in the middle of it on the X axis and on the very bottom of the Y axis. Its position is then set to that of the entity's, but with the UI element's offset factored in. Because the texture is set up to repeat itself, this allows us to represent ridiculous amounts of health:



When an entity's name is rendered, the `sf::Text` instance is first set up by changing the string and its origin is manipulated to be exactly in the middle. Since we want our information to be nicely stacked and not drawn on top of each other, checking if the health was rendered is necessary.

If the health component is present, the name's position is obtained from the `m_heartBar` data member. The Y value of that position is modified by subtracting the height of the health bar in order to render the player name on top. Otherwise, the name's position is set to match the entity with the offset included. The name background is then set up to be slightly larger than the text that it will be drawn behind and its origin is set to the exact center. The position of the name background is slightly offset by a single pixel from the position of the actual name. The values used here can be perfected by simply trying out different things and getting the feel for the best result.

Lastly, the background and the entity's name are drawn in that order on screen.

Network class and interpolation

Showing our entities simply appearing on the screen isn't satisfactory at all. Even if we get them to move, you will quickly notice that due to the delay between the server and client, players would look more like they're skipping across the screen, rather than walking. A little more work has to be done on the client side in order to smooth it out. For that, we're going to rely on something called interpolation. Consider the following illustration:



What is interpolation? It's an estimation between two known data points. There are many different types of interpolation out there, all with a different philosophy of use. For our purposes, interpolating data simply comes down to finding a weighted average between two values at a given time. In the preceding diagram, we have two snapshots representing different places in time. Interpolating helps us find the state of an entity somewhere in the middle of those two snapshots, and, in turn, smooths out their movement by adjusting attributes such as position, velocity, and acceleration based on the estimation, rather than actual snapshot data.

Finding a value at a specific point in time between two snapshots can be expressed this way:

$$\text{value}(t_x) = \left(\frac{\Delta \text{value}}{\Delta t} * (t_x - t_1) \right) + \text{value}(t_1)$$

A value we want to find at a given time, t_x , is simply the difference of the value between both snapshots divided by the difference in time, multiplied by the time that has passed since the first snapshot and then added to the value of the first snapshot. In code, it can be expressed like this:

```
template<class T>
inline T Interpolate(const sf::Int32& T1, const sf::Int32& T2,
    const T& T1_val, const T& T2_val, const sf::Int32& T_X)
{
    return (((T2_val - T1_val) / (T2 - T1)) * (T_X - T1)) + T1_val;
}
```

Having a few extra methods for actually handling the snapshot and time types, as well as comparing two snapshots together, would be useful:

```
void InterpolateSnapshot(const EntitySnapshot& l_s1,
    const sf::Int32& T1, const EntitySnapshot& l_s2,
    const sf::Int32& T2, EntitySnapshot& l_target,
    const sf::Int32& T_X);
```

```
bool CompareSnapshots(const EntitySnapshot& l_s1,
    const EntitySnapshot& l_s2, bool l_position = true,
    bool l_physics = true, bool l_state = true);
```

We're going to need some way of containing these snapshots, so it's time to define our data types:

```
using SnapshotMap = std::unordered_map<EntityId, EntitySnapshot>;
struct SnapshotDetails{
    SnapshotMap m_snapshots;
};
using SnapshotContainer = std::map<sf::Int32, SnapshotDetails>;
using OutgoingMessages = std::unordered_map<EntityMessage,
    std::vector<Message>>;
```

All of the snapshots are first stored with the entity ID being the key. The actual map itself is being held by a `SnapshotDetails` struct, which may prove to be useful later if we decide to add any additional snapshot information. All of the entity data is then stored in a map structure, where the timestamp of the snapshot is the key value. Notice that we're using a regular map here, as opposed to an unordered map. What's the benefit, you may ask. The regular map type may be a little bit slower, but it automatically sorts its entries by key. This means that newer snapshots will always go towards the end of the map. The reason why that's important will become apparent when we're performing entity interpolation.

The last data type we're going to need for the network class is some sort of container that holds outgoing messages we're going to send to the server. In this case, an unordered map works just fine.

So, what is our network system class going to look like? Let's take a look:

```
class S_Network : public S_Base{
public:
    S_Network(SystemManager* l_systemMgr);
    ~S_Network();

    void Update(float l_dT);
    void HandleEvent(const EntityId& l_entity,
        const EntityEvent& l_event);
    void Notify(const Message& l_message);

    void SetClient(Client* m_client);
    void SetPlayerID(const EntityId& l_entity);
```

```

    void AddSnapshot(const EntityId& l_entity,
        const sf::Int32& l_timestamp,
        EntitySnapshot& l_snapshot);
    void SendPlayerOutgoing();
    void ClearSnapshots();
private:
    void ApplyEntitySnapshot(const EntityId& l_entity,
        const EntitySnapshot& l_snapshot,
        bool l_applyPhysics);

    void PerformInterpolation();
    SnapshotContainer m_entitySnapshots;
    EntityId m_player;
    OutgoingMessages m_outgoing;
    Client* m_client;
    sf::Time m_playerUpdateTimer;
};

```

Apart from the normal methods a system has to implement, we have a few setters for registering a `Client` instance, as well as keeping track of the entity ID that our client is going to be controlling as a player. A few helper methods for adding a received entity snapshot, as well as sending out player messages to the server also exist to make life just a little bit easier. For our private method selection, we have a total of two: one for applying a specific snapshot to an entity and another for performing interpolation. This is met by a standard number of data members that are responsible for containing received snapshots, keeping track of the player ID, containing outgoing messages to the server before they're sent out, and having access to the `Client` instance. To top that off, we're going to use another `sf::Time` data type in order to keep track of time passage for sending player updates to the server.

Implementing the client network class

Before we get to actually implementing the network system, let's complete the last two functions related to interpolation and comparison of entity snapshots:

```

void InterpolateSnapshot(const EntitySnapshot& l_s1,
    const sf::Int32& T1, const EntitySnapshot& l_s2,
    const sf::Int32& T2, EntitySnapshot& l_target,
    const sf::Int32& T_X)
{
    l_target.m_direction = l_s2.m_direction;
    l_target.m_health = l_s2.m_health;
    l_target.m_name = l_s2.m_name;
    l_target.m_state = l_s1.m_state;
}

```

```
l_target.m_elevation = l_s1.m_elevation;

l_target.m_position.x = Interpolate<float>(
    T1, T2, l_s1.m_position.x, l_s2.m_position.x, T_X);
l_target.m_position.y = Interpolate<float>(
    T1, T2, l_s1.m_position.y, l_s2.m_position.y, T_X);

l_target.m_velocity.x = Interpolate<float>(
    T1, T2, l_s1.m_velocity.x, l_s2.m_velocity.x, T_X);
l_target.m_velocity.y = Interpolate<float>(
    T1, T2, l_s1.m_velocity.y, l_s2.m_velocity.y, T_X);

l_target.m_acceleration.x = Interpolate<float>(
    T1, T2, l_s1.m_acceleration.x, l_s2.m_acceleration.x, T_X);
l_target.m_acceleration.y = Interpolate<float>(
    T1, T2, l_s1.m_acceleration.y, l_s2.m_acceleration.y, T_X);
}
```

We begin by overwriting some values that don't need to be interpolated. Note that the direction, health, and name values are overwritten with the latest available information from the second entity snapshot, rather than the first. This provides an overall smoother feel to entity movement and interactions. For the rest of the snapshot data, we use our handy `Interpolate` function, which provides a smooth transition between the two updates.

It's also quite useful to have a function that can compare two snapshots together, so we can know if any data has changed. `CompareSnapshots` comes to the rescue here:

```
bool CompareSnapshots(const EntitySnapshot& l_s1,
    const EntitySnapshot& l_s2, bool l_position,
    bool l_physics, bool l_state)
{
    if (l_position && (l_s1.m_position != l_s2.m_position ||
        l_s1.m_elevation != l_s2.m_elevation))
    { return false; }
    if (l_physics && (l_s1.m_velocity != l_s2.m_velocity ||
        l_s1.m_acceleration != l_s2.m_acceleration ||
        l_s1.m_direction != l_s2.m_direction))
    { return false; }
    if (l_state && (l_s1.m_state != l_s2.m_state))
    { return false; }
    return true;
}
```

It's not really necessary to check every single aspect of a snapshot here. All we really care about is the positional, kinematic, and state information of the entity. Three additional Boolean arguments can also be provided, telling this function which data is relevant.

With this out of the way, we can finally begin implementing the network system class, starting, of course, with the constructor and destructor:

```
S_Network::S_Network(SystemManager* l_systemMgr)
    : S_Base(System::Network, l_systemMgr), m_client(nullptr)
{
    Bitmask req;
    req.TurnOnBit((unsigned int)Component::Client);
    m_requiredComponents.push_back(req);

    m_systemManager->GetMessageHandler()->
        Subscribe(EntityMessage::Move, this);
    m_systemManager->GetMessageHandler()->
        Subscribe(EntityMessage::Attack, this);
    m_playerUpdateTimer = sf::milliseconds(0);
}
```

Much like on the server class, we only care about the entities that have client components in this system. Messages for entity movement and attacks are also subscribed to in order to properly store them and update the server later on.

Next up, we have the Update method:

```
void S_Network::Update(float l_dT){
    if (!m_client){ return; }
    sf::Lock lock(m_client->GetMutex());
    m_playerUpdateTimer += sf::seconds(l_dT);
    if (m_playerUpdateTimer.asMilliseconds() >=
        PLAYER_UPDATE_INTERVAL)
    {
        SendPlayerOutgoing();
        m_playerUpdateTimer = sf::milliseconds(0);
    }
    PerformInterpolation();
}
```

First, a check is made to make sure we have a valid pointer to the client class. If so, we lock the client mutex and add time to the player update timer. The `SendPlayerOutgoing` method is then invoked and the timer is reset if enough time has passed to update the server. Lastly, we call the private helper method of this class, which is responsible for interpolating between snapshots. Keeping this functionality separate from the actual update loop leaves us with nicer looking code and allows early return while interpolating.

Handling the messages this system is subscribed to is quite simple, as you will see here:

```
void S_Network::Notify(const Message& l_message) {
    if (!HasEntity(l_message.m_receiver) ||
        l_message.m_receiver != m_player)
    {
        return;
    }
    if (l_message.m_type == (MessageType)EntityMessage::Attack &&
        m_outgoing.find(EntityMessage::Attack) != m_outgoing.end())
    {
        return;
    }
    m_outgoing[(EntityMessage)l_message.m_type].
        emplace_back(l_message);
}
```

At this point, all we care about is adding the message into our outgoing container, since we're not dealing with more complex types just yet. An additional check is performed in case an attack message is received. There really is no point of having multiple attack messages in this container at the same time, so the `Notify` method simply returns if an attack message is attempted to be inserted while one already exists in the container.

Next, we have some helper methods:

```
void S_Network::SetClient(Client* l_client) {m_client = l_client;}
void S_Network::SetPlayerID(const EntityId& l_entity) {
    m_player = l_entity;
}
void S_Network::AddSnapshot(const EntityId& l_entity,
    const sf::Int32& l_timestamp, EntitySnapshot& l_snapshot)
{
    sf::Lock lock(m_client->GetMutex());
    auto i = m_entitySnapshots.emplace(l_timestamp,
        SnapshotDetails());
    i.first->second.m_snapshots.emplace(l_entity, l_snapshot);
}
```

There's nothing too special going on here. One thing to note is that when a new snapshot is being added, the client mutex probably should be locked. Speaking of snapshots, let's look at how one could be applied to an entity:

```
void S_Network::ApplyEntitySnapshot(const EntityId& l_entity,
    const EntitySnapshot& l_snapshot, bool l_applyPhysics)
{
    ClientEntityManager* entities =
        (ClientEntityManager*)m_systemManager->GetEntityManager();
    C_Position* position = nullptr;
    C_Movable* movable = nullptr;
    S_Movement* movement_s = nullptr;
    S_State* state_s = nullptr;
    C_Health* health = nullptr;
    C_Name* name = nullptr;
    sf::Lock lock(m_client->GetMutex());
    if (position = entities->GetComponent<C_Position>(l_entity,
        Component::Position))
    {
        position->SetPosition(l_snapshot.m_position);
        position->SetElevation(l_snapshot.m_elevation);
    }
    if (l_applyPhysics){
        if (movable = entities->GetComponent<C_Movable>(l_entity,
            Component::Movable))
        {
            movable->SetVelocity(l_snapshot.m_velocity);
            movable->SetAcceleration(l_snapshot.m_acceleration);
        }
    }
    if (movement_s = m_systemManager->
        GetSystem<S_Movement>(System::Movement))
    {
        movement_s->SetDirection(l_entity,
            (Direction)l_snapshot.m_direction);
    }
    if (state_s = m_systemManager->
        GetSystem<S_State>(System::State))
    {
        state_s->ChangeState(l_entity,
            (EntityState)l_snapshot.m_state, true);
    }
    if (health = entities->GetComponent<C_Health>(l_entity,
        Component::Health))
```

```
{
    health->SetHealth(l_snapshot.m_health);
}
if (name = entities->GetComponent<C_Name>(l_entity,
    Component::Name))
{
    name->SetName(l_snapshot.m_name);
}
}
```

After we obtain a pointer to the entity manager and set up empty pointers to various components that the entity snapshot might contain information about, the client mutex is locked and we begin manipulating the component information carefully, by first attempting to retrieve a valid component address inside the `if` statements. This method also takes in a flag to let it know whether physics information, such as acceleration or velocity, should be applied, which can come in handy.

The following method is executed while updating the network system class, and it is responsible for sending player updates to the server:

```
void S_Network::SendPlayerOutgoing(){
    sf::Int32 p_x = 0, p_y = 0;
    sf::Int8 p_a = 0;

    for (auto &itr : m_outgoing){
        if (itr.first == EntityMessage::Move){
            sf::Int32 x = 0, y = 0;
            for (auto &message : itr.second){
                if (message.m_int == (int)Direction::Up){ --y; }
                else if (message.m_int == (int)Direction::Down){ ++y; }
                else if (message.m_int == (int)Direction::Left){ --x; }
                else if (message.m_int == (int)Direction::Right){ ++x; }
            }
            if (!x && !y){ continue; }
            p_x = x; p_y = y;
        } else if (itr.first == EntityMessage::Attack){ p_a = 1; }
    }

    sf::Packet packet;
    StampPacket(PacketType::PlayerUpdate, packet);
    packet << sf::Int8(EntityMessage::Move)
        << p_x << p_y << sf::Int8(Network::PlayerUpdateDelim);
    packet << sf::Int8(EntityMessage::Attack)
        << p_a << sf::Int8(Network::PlayerUpdateDelim);
}
```

```

    m_client->Send(packet);
    m_outgoing.clear();
}

```

We begin by setting up some local variables that are going to be holding the number of times our player has moved in the X and Y directions. A smaller variable is also set up for the attack state. The next step is to iterate over all outgoing messages and process each type individually. In a case of a `Move` type, every single one of them is counted. If an `Attack` message is found, the attack state is simply set to 1.

The last step is, of course, sending this information out. A new packet is then constructed and marked as a player update. The movement and attack state information is then fed into the packet. Notice that we're adding in the `PlayerUpdateDelim` value at the end of each update type. Enforcing specific communication rules as such decreases the chances of our server processing invalid or damaged data. Once the update packet is sent in, the outgoing message container is cleared for the next time.

Lastly, we arrive at the key method for ensuring smooth entity movement:

```

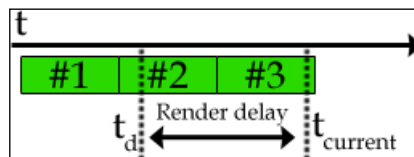
void S_Network::PerformInterpolation() {
    if (m_entitySnapshots.empty()) { return; }
    ClientEntityManager* entities =
        (ClientEntityManager*)m_systemManager->GetEntityManager();
    sf::Time t = m_client->GetTime();
    auto itr = ++m_entitySnapshots.begin();
    while (itr != m_entitySnapshots.end()) {
        if (m_entitySnapshots.begin()->first <=
            t.asMilliseconds() - NET_RENDER_DELAY &&
            itr->first >= t.asMilliseconds() - NET_RENDER_DELAY)
        {
            auto Snapshot1 = m_entitySnapshots.begin();
            auto Snapshot2 = itr;
            bool SortDrawables = false;
            for (auto snap = Snapshot1->second.m_snapshots.begin();
                snap != Snapshot1->second.m_snapshots.end(); )
            {
                if (!entities->HasEntity(snap->first)) {
                    if (entities->AddEntity(snap->second.m_type,
                        snap->first) == (int)Network::NullID)
                    {
                        std::cout << "Failed adding entity type: "
                            << snap->second.m_type << std::endl;
                        continue;
                    }
                }
                ApplyEntitySnapshot(snap->first, snap->second, true);
                ++snap;
            }
        }
    }
}

```

```
        continue;
    }
    auto snap2 = Snapshot2->second.m_snapshots.find(
        snap->first);
    if (snap2 == Snapshot2->second.m_snapshots.end()) {
        sf::Lock lock(m_client->GetMutex());
        entities->RemoveEntity(snap->first);
        snap = Snapshot1->second.m_snapshots.erase(snap);
        continue;
    }

    EntitySnapshot i_snapshot;
    InterpolateSnapshot(snap->second, Snapshot1->first,
        snap2->second, Snapshot2->first,
        i_snapshot, t.asMilliseconds() - NET_RENDER_DELAY);
    ApplyEntitySnapshot(snap->first, i_snapshot, true);
    if (!CompareSnapshots(snap->second, snap2->second,
        true, false, false))
    {
        SortDrawables = true;
    }
    ++snap;
}
if (SortDrawables) {
    m_systemManager->GetSystem<S_Renderer>
        (System::Renderer) ->SortDrawables();
}
return;
}
m_entitySnapshots.erase(m_entitySnapshots.begin());
itr = ++m_entitySnapshots.begin();
}
}
```

First and foremost, we must deal with the possibility of our client not having any snapshots at all. If that happens, this method is returned from immediately. If we have snapshots available, the next step is iterating over the snapshot container and finding two snapshots that we're currently between (time wise). Normally, this wouldn't be a likely outcome, but keep in mind that we're rendering things slightly in the past:



The benefit of rendering slightly in the past is that we will actually have more data that has arrived from the server, which in turn will allow us to smooth it out and provide nicer entity movement. This wouldn't be possible if we simply rendered everything in real time. This delay is represented by the `NET_RENDER_DELAY` macro.

Once we find the pair of snapshots that we're looking for, a local variable called `SortDrawables` is set up to keep track of whether or not we need to worry about re-sorting drawable components to represent depth correctly. All of the entities from the first (earlier) snapshot are then iterated over. Our first concern is making sure that an entity that exists in the snapshot also exists on our client. If it doesn't, a new entity is created from the type that the snapshot provides. All of its information is then applied to the newly created entity and we skip the current iteration of the snapshot loop as there's no need to interpolate anything.

The next step is making sure that the entity that exists in the earlier snapshot also exists in the later one, so an attempt to find it in the second snapshot container is made. Provided the entity has not been found, the client mutex is locked and the entity is removed from our client, prior to actually being erased from the snapshot container as well. The current iteration is then skipped, as we have no reason to interpolate once again.

If all of these checks yield no reason for us to skip an iteration, a new instance of `EntitySnapshot` is created. This is going to be our target for holding interpolated data. `InterpolateSnapshot` is then called with both snapshots and their time values, as well as the target snapshot and the current time *with the interpolation delay* factored in is passed in as arguments. After the target snapshot is filled in with the interpolated data, it is applied to the current entity. We also want to compare both snapshots we're interpolating between and set the `SortDrawables` variable to `true` if they have different positions. After all of the entity interpolation code, this variable is checked and the system renderer is instructed to re-sort the drawable components if it was indeed set to `true` at some point.

One last thing to take away from this is that if the time checking conditional in the very first loop ends up not being satisfied, the first element in the snapshot container is erased and the iterator is reset to point to the second value in it, ensuring a proper disposal of irrelevant snapshots.

Client entity and system management

Quite predictably, we're going to have different types of components and systems available on the client side than the server side, starting with component types:

```
ClientEntityManager::ClientEntityManager(SystemManager* l_sysMgr,
TextureManager* l_textureMgr): EntityManager(l_sysMgr),
m_textureManager(l_textureMgr)
{
    AddComponentType<C_Position>(Component::Position);
    AddComponentType<C_State>(Component::State);
    AddComponentType<C_Movable>(Component::Movable);
    AddComponentType<C_Controller>(Component::Controller);
    AddComponentType<C_Collidable>(Component::Collidable);
    AddComponentType<C_SpriteSheet>(Component::SpriteSheet);
    AddComponentType<C_SoundEmitter>(Component::SoundEmitter);
    AddComponentType<C_SoundListener>(Component::SoundListener);
    AddComponentType<C_Client>(Component::Client);
    AddComponentType<C_Health>(Component::Health);
    AddComponentType<C_Name>(Component::Name);
    AddComponentType<C_UI_Element>(Component::UI_Element);
}
```

After making sure all client-relevant component types are registered, let's implement our own version of loading an entity here, as it involves manipulating the renderable components it may have:

```
int ClientEntityManager::AddEntity(
    const std::string& l_entityFile, int l_id)
{
    ...
    while (std::getline(file, line)){
        ...
        } else if (type == "Component"){
            ...
            keystream >> *component;
            if (component->GetType() == Component::SpriteSheet){
                C_SpriteSheet* sheet = (C_SpriteSheet*)component;
                sheet->Create(m_textureManager);
            }
        }
    }
    ...
}
```

We have already seen this code in previous chapters, but it's still fair to emphasize that the highlighted snippet does not exist on the server side at all, yet is necessary here.

Next, the client's version of a system manager:

```
class ClientSystemManager : public SystemManager{
public:
    ClientSystemManager(TextureManager* l_textureMgr,
        FontManager* l_fontMgr);
    ~ClientSystemManager();

    TextureManager* GetTextureManager();
    FontManager* GetFontManager();
    void Draw(Window* l_wind, unsigned int l_elevation);
private:
    TextureManager* m_textureMgr;
    FontManager* m_fontMgr;
};
```

Naturally, the only additions we have here are, once again, related to graphics. We wouldn't need to draw anything on the server side, but it's necessary here.

The constructor of our client system manager handles adding systems that are relevant to the client performing as intended:

```
ClientSystemManager::ClientSystemManager(
    TextureManager* l_textureMgr, FontManager* l_fontMgr)
    : m_textureMgr(l_textureMgr), m_fontMgr(l_fontMgr)
{
    AddSystem<S_State>(System::State);
    AddSystem<S_Control>(System::Control);
    AddSystem<S_Movement>(System::Movement);
    AddSystem<S_Collision>(System::Collision);
    AddSystem<S_SheetAnimation>(System::SheetAnimation);
    AddSystem<S_Network>(System::Network);
    AddSystem<S_Sound>(System::Sound);
    AddSystem<S_Renderer>(System::Renderer);
    AddSystem<S_CharacterUI>(System::Character_UI);
}
```

Note the placement of the network system here. The order of adding these systems directly dictates the order in which they are updated. We don't want our network system sending or receiving any data before we get a chance to process our own.

Naturally, getters for texture and font managers would be useful on this side:

```
TextureManager* ClientSystemManager::GetTextureManager() {
    return m_textureMgr;
}
FontManager* ClientSystemManager::GetFontManager() {
    return m_fontMgr;
}
```

Lastly, we have a few systems that need to render something on screen:

```
void ClientSystemManager::Draw(Window* l_wind,
    unsigned int l_elevation)
{
    auto itr = m_systems.find(System::Renderer);
    if(itr != m_systems.end()){
        S_Renderer* system = (S_Renderer*)itr->second;
        system->Render(l_wind, l_elevation);
    }
    itr = m_systems.find(System::Character_UI);
    if (itr != m_systems.end()){
        S_CharacterUI* ui = (S_CharacterUI*)itr->second;
        ui->Render(l_wind);
    }
}
```

After the renderer system draws all of the entities on screen, we want to overlay their names and health graphics on top of that.

Putting the pieces into place

Because all of the networking and action is going to take place solely within the confines of the game state, that's the main class we're going to adjust, starting with the header file:

```
class State_Game : public BaseState{
    ...
private:
    Map* m_gameMap;
    int m_player;
    Client* m_client;
};
```

After making sure that the game state has a pointer to a `Client` instance, we must provide a way for the game to handle incoming packets:

```
void State_Game::HandlePacket(const PacketID& l_id,
    sf::Packet& l_packet, Client* l_client)
{
    ClientEntityManager* emgr = m_stateMgr->
        GetContext()->m_entityManager;
    PacketType type = (PacketType)l_id;
    if (type == PacketType::Connect){
        sf::Int32 eid;
        sf::Vector2f pos;
        if (!(l_packet >> eid) || !(l_packet >> pos.x) ||
            !(l_packet >> pos.y))
        {
            std::cout << "Faulty CONNECT response!" << std::endl;
            return;
        }
        std::cout << "Adding entity: " << eid << std::endl;
        m_client->GetMutex().lock();
        emgr->AddEntity("Player", eid);
        emgr->GetComponent<C_Position>
            (eid, Component::Position)->SetPosition(pos);
        m_client->GetMutex().unlock();
        m_player = eid;
        m_stateMgr->GetContext()->m_systemManager->
            GetSystem<S_Network>(System::Network)->SetPlayerID(m_player);
        emgr->AddComponent(eid, Component::SoundListener);
        return;
    }

    if (!m_client->IsConnected()){ return; }
    switch (type){
    case PacketType::Snapshot:
    {
        sf::Int32 entityCount = 0;
        if (!(l_packet >> entityCount)){
            std::cout << "Snapshot extraction failed."
                << std::endl;
            return;
        }
        sf::Lock lock(m_client->GetMutex());
        sf::Int32 t = m_client->GetTime().asMilliseconds();
        for (unsigned int i = 0; i < entityCount; ++i){
```

```
        sf::Int32 eid;
        EntitySnapshot snapshot;
        if (!(l_packet >> eid) || !(l_packet >> snapshot)){
            std::cout << "Snapshot extraction failed."
                << std::endl;
            return;
        }
        m_stateMgr->GetContext()->m_systemManager->
            GetSystem<S_Network>(System::Network)->
                AddSnapshot(eid, t, snapshot);
    }
    break;
}
case PacketType::Disconnect:
{
    m_stateMgr->Remove(StateType::Game);
    m_stateMgr->SwitchTo(StateType::MainMenu);
    std::cout << "Disconnected by server!" << std::endl;
    break;
}
case PacketType::Hurt:
{
    EntityId id;
    if (!(l_packet >> id)){ return; }
    Message msg((MessageType)EntityMessage::Hurt);
    msg.m_receiver = id;
    m_stateMgr->GetContext()->m_systemManager->
        GetMessageHandler()->Dispatch(msg);
    break;
}
}
```

First, we handle the connect packet that the server sends back to us after the client tries to reach it. If the entity ID and position were successfully extracted from the packet, the client mutex is locked while the player entity is added and its position is updated. The entity ID of our player is then stored in the `m_player` data member and passed in to our network system, which needs it. Note the very last line of code in this segment before we return. After the entity is successfully constructed, we're adding in a sound listener component to it. Naturally, there can only be one single sound listener on the client side, which is our player. This means that the `player.entity` file *does not* have its own sound listener component anymore. Instead, it must be added here in order to have correct audio positioning.

Next, if our client is already connected to the server, we're ready to process snapshot, hurt, and disconnect packets. If a snapshot is received, we first attempt to read the number of entities it contains and return if the reading fails. The client mutex is then locked and the current time is obtained in order to maintain continuity of entity snapshots. A new `for` loop is then constructed to run for each individual entity in the packet and extract its ID and snapshot data, which in turn is added to the network system for later processing.

If a disconnect packet is received from the server, we simply remove the game state and switch back to the main menu. Also, upon receiving a hurt packet, the entity ID in it is extracted and a `Hurt` message that is to be received by that entity is created and sent out.

Now, it's time to adjust the existing methods of our game state in order to have it try to establish a connection to the server upon creation:

```
void State_Game::OnCreate() {
    m_client->Setup(&State_Game::HandlePacket, this);
    if (m_client->Connect()) {
        m_stateMgr->GetContext()->m_systemManager->
            GetSystem<S_Network>(System::Network)->SetClient(m_client);
        ...
        evMgr->AddCallback(StateType::Game, "Player_Attack",
            &State_Game::PlayerAttack, this);
        ...
    } else {
        std::cout << "Failed to connect to the game server!"
            << std::endl;
        m_stateMgr->Remove(StateType::Game);
        m_stateMgr->SwitchTo(StateType::MainMenu);
    }
}
```

First, the client's packet handler is assigned. We then attempt to connect to the server with whatever IP and port information exist inside the client class at this point. If the connection attempt was successful, we can start initializing our data members and add callbacks, one of which is a callback to a new method that handles the player attack button being pressed. If the connection wasn't successful, the game state is removed and we switch back to the main menu state instead.

If the game state is removed, some cleanup is in order:

```
void State_Game::OnDestroy() {
    m_client->Disconnect();
    m_client->UnregisterPacketHandler();
    S_Network* net = m_stateMgr->GetContext()->
        m_systemManager->GetSystem<S_Network>(System::Network);
    net->ClearSnapshots();
    net->SetClient(nullptr);
    net->SetPlayerID((int)Network::NullID);
    ...
    evMgr->RemoveCallback(StateType::Game, "Player_Attack");
    ...
}
```

In addition to the rest of the code that cleans up the game state, we must now also disconnect from the server and unregister the packet handler that is being used by the client class. The network system is also cleared of all snapshots it may currently hold, as well as any player information and pointers to the client class. The player attack callback is also removed here.

Naturally, we're going to want to alter the Update method of the game state a little as well:

```
void State_Game::Update(const sf::Time& l_time) {
    if (!m_client->IsConnected()) {
        m_stateMgr->Remove(StateType::Game);
        m_stateMgr->SwitchTo(StateType::MainMenu);
        return;
    }
    SharedContext* context = m_stateMgr->GetContext();
    UpdateCamera();
    m_gameMap->Update(l_time.asSeconds());
    {
        sf::Lock lock(m_client->GetMutex());
        context->m_systemManager->Update(l_time.asSeconds());
    }
}
```

The connection status of our client is first checked. Not being connected means we get to exit the game state and switch back to the main menu once again. Otherwise, we continue on with the updating. Note the curly brackets surrounding the system manager update call. They create a scope for any variables defined inside, which is useful for locking the client mutex with a `sf::Lock` instance, as it will fall out of scope once we're outside the brackets, in turn unlocking it.

Drawing things on screen also needs a slight adjustment:

```
void State_Game::Draw() {
    if (!m_gameMap) { return; }
    sf::Lock lock(m_client->GetMutex());
    for (int i = 0; i < Sheet::Num_Layers; ++i) {
        m_gameMap->Draw(i);
        m_stateMgr->GetContext()->m_systemManager->
            Draw(m_stateMgr->GetContext()->m_wind, i);
    }
}
```

The only addition here is the client mutex lock right before we draw entities on different elevations in a `for` loop. We don't want another thread to manipulate any data that we may be currently accessing.

Lastly, the player attack button being pressed needs to be handled like this:

```
void State_Game::PlayerAttack(EventDetails* l_details) {
    Message msg((MessageType)EntityMessage::Attack);
    msg.m_receiver = m_player;
    m_stateMgr->GetContext()->m_systemManager->
        GetMessageHandler()->Dispatch(msg);
}
```

It's quite simple. When an attack key is pressed, the entity component system has a new attack message sent to it. Our network system is subscribed to this message type and adds it to the player update container, which is going to be sent out to the server at a specific interval.

Main menu adjustments

Our client-server setup is now functional, but we are missing one more small addition in order to really make it work. We have no way of putting in our server information! Let's fix that by modifying the main menu interface file:

```
Interface MainMenu MainMenu.style 0 0 Immovable NoTitle "Main menu"
Element Label Title 100 0 MainMenuTitle.style "Main menu:"
Element Label IpLabel 0 32 DefaultLabel.style "IP:"
Element TextField IP 18 32 MainMenuTextfield.style "127.0.0.1"
Element Label PortLabel 150 32 DefaultLabel.style "Port:"
Element TextField PORT 175 32 MainMenuTextfield.style "5600"
Element Label NameLabel 50 56 DefaultLabel.style "Nickname:"
Element TextField Nickname 105 56 MainMenuTextfield.style "Player"
Element Label Play 0 80 MainMenuLabel.style "CONNECT"
```

```
Element Label Disconnect 0 116 MainMenuLabel.style "DISCONNECT"
Element Label Credits 0 152 MainMenuLabel.style "CREDITS"
Element Label Quit 0 188 MainMenuLabel.style "EXIT"
```

Quite a few new elements are added to the main menu here. We have three new text fields and some text labels that go next to them to let the user know what they're for. This is how server information, as well as the player nickname, is going to be entered. Let's make this happen by adding a few callbacks for the new buttons:

```
void State_MainMenu::OnCreate() {
    SetTransparent(true); // Transparent for rendering.
    SetTranscendent(true); // Transcendent for updating.
    ...
    eMgr->AddCallback(StateType::MainMenu, "MainMenu_Play",
        &State_MainMenu::Play, this);
    eMgr->AddCallback(StateType::MainMenu, "MainMenu_Disconnect",
        &State_MainMenu::Disconnect, this);
    eMgr->AddCallback(StateType::MainMenu, "MainMenu_Quit",
        &State_MainMenu::Quit, this);
}

void State_MainMenu::OnDestroy() {
    ...
    gui->RemoveInterface(StateType::MainMenu, "MainMenu");
    eMgr->RemoveCallback(StateType::MainMenu, "MainMenu_Play");
    eMgr->RemoveCallback(StateType::MainMenu, "MainMenu_Disconnect");
    eMgr->RemoveCallback(StateType::MainMenu, "MainMenu_Quit");
}
```

To make the main menu feel interactive, we're going to want to update this interface each time the menu state is activated:

```
void State_MainMenu::Activate() {
    GUI_Interface* menu = m_stateMgr->GetContext()->
        m_guiManager->GetInterface(StateType::MainMenu, "MainMenu");
    if (m_stateMgr->HasState(StateType::Game)) {
        // Resume
        menu->GetElement("Play")->SetText("Resume");
        menu->GetElement("Disconnect")->SetActive(true);
        menu->GetElement("IP")->SetActive(false);
        menu->GetElement("PORT")->SetActive(false);
        menu->GetElement("IpLabel")->SetActive(false);
        menu->GetElement("PortLabel")->SetActive(false);
        menu->GetElement("NameLabel")->SetActive(false);
        menu->GetElement("Nickname")->SetActive(false);
    }
```

```

    } else {
        // Play
        menu->GetElement("Play")->SetText("CONNECT");
        menu->GetElement("Disconnect")->SetActive(false);
        menu->GetElement("IP")->SetActive(true);
        menu->GetElement("PORT")->SetActive(true);
        menu->GetElement("IpLabel")->SetActive(true);
        menu->GetElement("PortLabel")->SetActive(true);
        menu->GetElement("NameLabel")->SetActive(true);
        menu->GetElement("Nickname")->SetActive(true);
    }
}

```

Depending on whether a game state exists or not, we set up the elements in our interface to reflect the current state of our connection.

Lastly, let's look at the callback methods of both connect and disconnect buttons:

```

void State_MainMenu::Play(EventDetails* l_details){
    if (!m_stateMgr->HasState(StateType::Game)){
        GUI_Interface* menu = m_stateMgr->GetContext()->
            m_guiManager->GetInterface(StateType::MainMenu, "MainMenu");
        std::string ip = menu->GetElement("IP")->GetText();
        PortNumber port = std::atoi(
            menu->GetElement("PORT")->GetText().c_str());
        std::string name = menu->GetElement("Nickname")->GetText();
        m_stateMgr->GetContext()->m_client->
            SetServerInformation(ip, port);
        m_stateMgr->GetContext()->m_client->SetPlayerName(name);
    }
    m_stateMgr->SwitchTo(StateType::Game);
}

void State_MainMenu::Disconnect(EventDetails* l_details){
    m_stateMgr->GetContext()->m_client->Disconnect();
}

```

The first check in the `Play` method is made to ensure the text field information is properly passed in to where it needs to go. Because we have the same button that's going to be pressed to both connect to the server and switch back to the game state once it exists, making sure the client instance's server and player name information is updated is important. We then switch to the game state, which could either mean that it has to be created, at which time the information we just passed in is used, or that it's simply brought back to being the dominant application state.

The disconnect button callback only invokes the client's `Disconnect` method, which in turn results to the game state terminating itself.

With that, we have a fully functional 2D multiplayer game where players can attack one another!



Summary

Congratulations! You have made it to the end! It has been quite a journey to take. With nothing more than some basic tools and concentrated effort, we have managed to create a small world. It may not have that much content in it, but that's where you come in. Just because you are done reading this book doesn't mean that either one of the three projects we covered is finished. In fact, this is only the beginning. Although we have covered a lot, there's still a plethora of features that you can implement on your own, such as different types of enemies, selectable player skins for the last project, magic and ranged attacks, animated map tiles, map transitions for the last project, a chat system, levels and experience for our RPG, and much more. Undoubtedly, you must have your own ideas and mechanics in mind that should instead be brought forth and realized in your games. Don't stop now; keep the flow going and get to coding!

Thank you so much for reading, and remember that ultimately whatever becomes of the world we created is in your hands, so make it a good one. Goodbye!